REL301m	FPT Education FPT UNIVERSITY	Final Assignment

TetrisRL: An Evolution Strategy Approach to AI Game Agents

by
Nguyen Anh Hao
SE171792

1. Introduction

Objective

The main objective of this project is to develop and evaluate an AI Agent for playing Tetris using an evolution strategies (ES).

Motivation

First release since 1984, Tetris remains one of the most popular and enduring games in history, celebrated for its simple yet addictive gameplay that has captivated millions around the world. Despite its straightforward mechanics, the game challenges the limits of human reaction time and decision-making, especially as the pace accelerates.

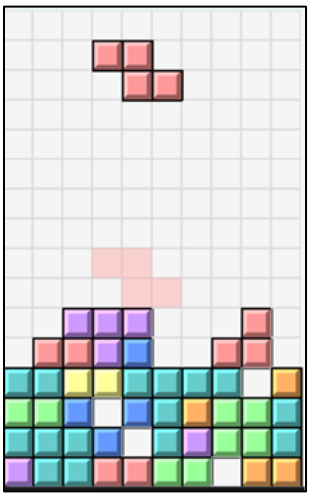


Figure: Tetris gameplay

2. Literature Review

Tetris AI – The (Near) Perfect Bot – YiYuan Lee (2013)

The blog post introduced a fairly efficient approach to training a tetris AI model: using a heuristic function and genetic algorithms (GA) appoarch. The results published by the author seem very promising: the model has played continuously for 2 weeks without losing. Unfortunately, the source code has been hidden by the author and only the source code of the Java webapp with a fixed set of weights is left. Source: https://codemyroad.wordpress.com/2013/04/14/tetris-ai-the-near-perfect-player/

3. TetrisRL

Based on the idea presented in the blog, I started the journey of rebuilding the entire environment, agent, algorithm from scratch.

Environment

The Tetris game environment was built in 4 parts:

1. **Piece:**
    - The available Tetris shapes (I, O, T, S, Z, J, L) are defined in a dictionary with their rotation matrices. Each shape has one or more rotations.

- Methods *rotate()* and *rotate_counterclockwise()* allow the piece to change its orientation, *move(dx, dy)* method updates the piece's position on the grid.
- *get_cells()* computes the absolute positions on the grid occupied by the piece.
- *clone()* method provides a way to create a duplicate of the piece, which is particularly useful when simulating moves or cloning the environment state.

2. **Grid:**
    - A 2D grid with specified dimensions, each cell is assigned to 0 (empty cell) or 1 (has block).
    - *is_valid_position(piece)* checks whether the cells are ready to place a piece in, *place_piece(piece)* marks the grid cells based on the piece's position.
    - *clear_lines()* identifies, counts and removes fully occupied rows (no empty cell), then shifts the remaining rows down.

3. **Random Piece Generator:**
    - mode= *'classic'*: a generator that uses the Tetris random algorithm called "7-bag generator". 7 piece shapes are shuffled and put into a bag, whenever the environment calls for a new piece, the generator will pop out a piece shape. Until there are no more pieces to pop out, the generator will fill the bag. This mode will ensure that within 14 moves there will be exactly 2 duplicates of each piece shape.
    - Mode= *'random'*: a completely random generator that chooses piece shape using *random.choice()*. This mode increases the difficulty of the game as it is possible to encounter difficult piece shapes (like 'S' or 'Z') multiple times in a row.
    - A random seed is set to ensure that the evaluation of the performance of different agents is objective.
4. **Game mechanism:**
    - *reset()* and *clone()* allow reinitializing the game or duplicating the current state. The cloning functionality ensures that both the grid and the pieces are duplicated, which is useful for simulations or move evaluations.
    - *move_piece(), rotate_piece(), drop_piece(), and hard_drop()* handle the piece movement and rotation logic, including collision detection and reverting invalid moves.
    - *new_piece()* and *update()* are used to continuously update the game state.
    - *get_possible_moves()* and *simulate_moves()* are created to support AI decision-making.

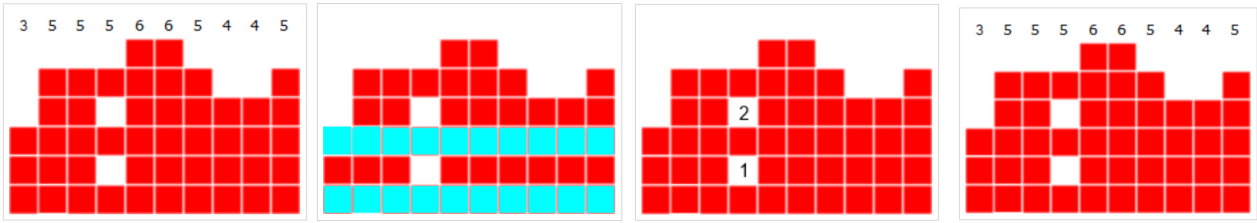
Agent

1. **The heuristic function (reward function):**

According to Yiyuan Lee, this Tetris game as 4 important characteristics: aggrerate heights, cleared lines, holes and bumpiness.

- **Aggrerate heights:** define by the sum of the height of each columns.
- **Cleared lines:** define by the number of completed lines in the grid.
- **Holes:** define as an empty space such that there is at least one tile in the same column above it.

- **Bumpiness:** define by computing the sum of the absolute differences between all two adjacent columns.



Aggregate Height = 48 | Complete lines = 2 | Number of holes = 2 | Bumpiness = 6

The reward of a state will be calculated by taking a linear combination of four heuristics above:

$$reward = \omega_1 \times (aggrerate_heights) + \omega_2 \times (cleared_lines) + \omega_3 \times (holes) + \omega_4 \times (bumpiness)$$

with $\omega_1, \omega_2, \omega_3, \omega_4$ are constant parameters

2. **Choose action strategies**

There are 2 mode *'normal'* and *'promax'* represent 2 strategies for agent when chosing action:

- *'normal'*: agent will get all possible moves from a state, calculate the reward for each possible move and choose the move that gain max reward.
- *'promax'*: this mode allow agent to find the best move by considering both the current piece and the next piece. For each moves in the current piece, simulate all possible moves of the next piece and choose the action that maximize the total reward (current + next piece).
- For the *'normal'* mode, the computational complexity is $O(n)$,compare to $O(n^2)$ of the *'promax'* mode.

Evolution Strategies

I have updated the Genetic Algorithm from the refence to the newer method, the Evolution Strategies, which have quite similar ideas but different implementations.

Evolution Strategies (ES) summary:

1. Create a 4D vector $[\omega_1, \omega_2, \omega_3, \omega_4]$ as baseline. Normalization on the 4-dimensional unit sphere.
2. Apply Gaussian noises to the baseline to create a population.
3. Each candidate in population plays simulate games .
4. Evaluate candidates rewards and update the baseline.
5. Loop the operation 2,3,4 until n-generations.
6. Evaluate training results, hyperparameter fine-tuning.

The training process was seperated into 2 phases: exploration and exploitation

1. **Exploration:** high sigma and learning rate to encourage exploration. The baseline is updated by:

$$\omega = \omega + \left(\frac{\alpha}{\sigma . n}\right) \times [\aleph(0,1) \cdot Z_{reward}]$$

$\omega: weights(baseline),$
$\alpha: learning\ rate,$
$\sigma: noise\ rate,$
$n: population,$

$\aleph(0,1)$: $Gaussian\ noises$,
Z_{reward}: $standardized\ rewards$

2. **Exploitation:** very small sigma and learning rate for better exploitation. The baseline now is updated by 'elite candidate strategy' – choose the candidate that has best performance to the the new baseline.

4. Agent Performance

To summarize the results of the entire project, I will present a performance comparison of the five models: base (starting point), first_explore as explore, first_exploit as exploit, best_model as best and ref (YiYuan Lee model).
With 2 generator option (random/classic) and 2 choose action strategy (normal/promax), each model has run 100 games 4 times and the agents' performances are collected as follow:

Agent	Generator	Mode	Max score	Min score	Average score	Max moves	Average moves
base	random	normal	144	18	66	402	208
base	random	promax	221	21	90	595	267
base	classic	normal	277	23	81	734	245
base	classic	promax	329	35	104	864	302
explore	random	normal	338	34	133	890	375
explore	random	promax	439	19	138	1140	389
explore	classic	normal	423	41	170	1099	468
explore	classic	promax	664	33	158	1704	436
exploit	random	normal	1570	26	452	3967	1171
exploit	random	promax	--	--	--	--	--
exploit	classic	normal	20970	94	3568	52469	8961
exploit	classic	promax	--	--	--	--	--
best	random	normal	2266	35	486	5710	1255
best	random	promax	--	--	--	--	--
best	classic	normal	21659	65	3887	54267	9513
best	classic	promax	--	--	--	--	--
ref	random	normal	1916	24	455	4834	1178
ref	random	promax	--	--	--	--	--
ref	classic	normal	20968	113	3796	52464	9532
ref	classic	promax	--	--	--	--	--

--: cannot evaluate due to hardware limitation (may cause by computational complexity or endless gameplay).
The table shown us that best_model perform better than reference but not significant. In term of using 'promax' mode, I was not able to wait until the testing process end, so I can assume that it was closed to a 'perfect' agent which undefeatable as Yiyuan Lee claimed in his blog.